

Telex Machine Log

<https://claude.ai/share/f1fb7a69-53eb-471e-881a-1554d5a5d39c>

Ship Steering Log

<https://claude.ai/share/b2522e4a-1c04-46ac-9b7a-cc5bb043f585>

ADR Log

<https://claude.ai/share/7a88fd34-ba7c-4e08-bd84-0e09c5bf8daf>

Claude Code Log

Prompt: Can you rebuild the Telex machine functionality to:

- Be random. Meaning, the Telex machine always sits stationary on the desk. Randomly, there will be a flashing light to indicate an incoming message.
- The Telex machine prints a small piece of paper, which does not display the message.
- Clicking on it does not open a separate tab. Instead, it simply enlarges the piece of paper to become legible. This can then be read.

Response: I've rebuilt the teleprinter on the helm and tested it in headless Chrome. The full flow works with no console errors.

****How it works now****

- ****Stays on the desk.**** The machine is just scenery now: hovering does nothing and clicking it doesn't open the full-screen view anymore.
- ****Random signals.**** The first signal arrives 8–20 seconds after the page loads. Each later one comes 40–120 seconds after you put the last slip down, so you never miss one. When a signal comes in, the lamp flashes and keeps flashing until you pick up the slip.
- ****Unreadable slip.**** A small strip of paper feeds out of the top of the machine a line at a time. From the helm it only shows rows of grey bars, one per printed line, with no readable text. The last slip stays in the machine until the next one replaces it, so you can re-read it.
- ****Enlarges in place.**** Clicking the slip grows it from where it sits into a full-size sheet over the helm, and the message is legible there. "Put it down", Esc, or clicking the dark background shrinks it back into the machine.

The messages still come from the existing files in `Telex Machine Demo/signals/`, and it reads the ship name and sign-off from `config.js`. Each signal is picked at random, and the same one never comes up twice in a row. I left the standalone Telex Machine Demo untouched.

****Changes****

- [shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html): the teleprinter markup is new, the iframe overlay is replaced by a reading sheet, and the timing settings are in the Tuning block (`SIGNAL\_FIRST`, `SIGNAL\_GAP`, `LINE\_FEED`).
- [shipstyle.css](Ship%20Functionality%20Demo/shipstyle.css): new styles for the slip feeding out, the lamp flashing and the sheet; the old iframe styles are removed.

****Things to know****

- The slip prints on its own as soon as the lamp lights; there's no button to press first. If you'd rather the player click the machine to start printing, that's a small change.
- The message-formatting code is copied from `telex.js` rather than shared with it, because `telex.js` only runs on its own page. If you change how signals are formatted, you'll need to update both copies.
- To test without waiting, run `TELEX.incoming()` in the browser console to bring a signal in straight away.

I haven't committed anything.

Prompt: Edit background visuals to create small waves in the sea, as well as rework some of the backgrounds. Add a large land border on one side of the map, which cannot be reached.

Response: The sea now has small waves, the backgrounds are reworked, and there's a coastline along the east side of the map that you can't reach. I tested it in headless Chrome by steering toward the coast, looking through the binoculars, and sailing into the shoals; there were no console errors.

****Small waves****

- Thousands of small wave crests are drawn in perspective: large near the bow, fine and packed tight toward the horizon.
- They roll toward you as the boat moves, sweep sideways as you turn, and drift with the wind. They catch the light more toward the sun's bearing, and now and then one breaks into a fleck of whitecap.
- They also show through the binoculars, where they move correctly whichever way you're looking.

****The coast****

- A continuous French coastline runs north–south about 2.6 km east of the start. From front to back it has surf, a beach, dunes, a wooded belt, and hazy hills behind.
- There's a town with rooftops and belfries, with a smoke plume from Dunkirk drifting out over the water.
- It fades into haze with distance and looks right at any heading or binocular power.

- ****It can't be reached.**** Shoals hold the boat 450 m offshore, and the boat slides along them rather than stopping. A red "Shoal water · bear away to port" plate lights on the dash as you get close.
- Nothing out on the water (buoys, islands, ships) ever appears standing on the land.

****Other background changes****

- The windscreen sky now shows the same drifting clouds as the binoculars; before, it was a plain gradient.
- I removed the two small "land" shapes floating in the open sea, since the coast replaces them.

****Frame rate:**** the new drawing costs something. In headless Chrome without a GPU, the windscreen went from about 100 to about 76 frames per second and the binoculars from about 72 to about 66. A normal machine with a GPU should notice less.

All the settings are grouped near the top of the script, under "The coast" and "Wavelets", in [shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html). For example, `COAST\_X` moves the shore, `COAST\_OFF` sets how close the shoals let you come, and `TOWN` and `SMOKE` place the town and its plume. The styles changed in [shipstyle.css](Ship%20Functionality%20Demo/shipstyle.css).

Nothing is committed yet; that includes the earlier teleprinter changes.

Prompt: Change the view of the horizon to be a small, subtle rising and lowering wave, instead of a static line.

Response: The horizon is no longer a ruled line. The far swell now lifts and settles gently along it, in the windscreen and through the binoculars. I checked it with zoomed screenshots taken two seconds apart, and the edge moves between them.

- ****How it moves:**** it's three overlapping slow swells with different lengths and speeds, so the pattern never visibly repeats. They're tied to compass bearing, so the swell slides past as you turn instead of being painted onto the glass.
- ****How big:**** at most about 0.07° high, which is 1–3 pixels in the windscreen. The binoculars magnify it along with everything else, so there it reads as a slow, low roll. It's tinted to match the haze at the horizon in each view, so the edge stays soft rather than a hard outline.
- ****Settings:**** `SKYLINE` in the Wavelets settings of [shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html) sets the height; raise it for a livelier sea. If the player has reduced motion turned on, it holds still.

****A fix to last round's coast:**** the close-ups showed a pale, hard-edged rectangle at the right of the horizon when heading north. It was the coast seen almost end-on 60+ km away, cut off sharply where the drawing stopped. Distant land now sinks over the curve of the Earth, so the

far coast tapers smoothly into the horizon instead. The near coast is effectively unchanged, since the drop there is under a metre.

Nothing is committed yet, including the teleprinter and coast work from earlier.

Prompt: Here's what I aim to add:

- A simple x/y coordinate system to the dashboard to show the view where they are in the world map. For simplicity, use the starting location as (0,0).
- Moving around the map will change the ship's coordinate locations.
- A small display, similar to the Telex machine, next to the compass. This will display the ship's current coordinates.

Response: The ship now has a position indicator. It sits on the dash just to starboard of the compass and shows how far the ship has moved from where it started.

I checked it with headless Edge screenshots at desktop, phone (390×844) and portrait tablet (820×1000) sizes. The screenshots show it placed correctly and the Y value going up as the ship sails north from the start. I haven't steered it around by hand in a browser.

****Coordinates****

- The starting location is (0, 0). Internally the boat starts at (0, -80) m, so I store that spot as the origin and show everything relative to it.
- Units are whole metres. X is east and Y is north, which matches how the game already tracks the boat. At 15 knots the numbers change about 7–8 times a second, so you can see the ship moving.
- The values are signed and padded to four digits, like ``+0123`` and ``-0045``. They grow to five digits past 10 km.
- The display updates each frame alongside the compass, but only changes the text when a value changes.
- It shows where the ship actually is, not a wrapped value. The sea scenery repeats every 4 km, but the coordinates keep counting. Near the French coast, where the shoals stop the ship, X stops growing.

****Display****

- It's a small instrument in the same olive enamel as the Telex, with a brass "POSITION" nameplate. X and Y are shown in amber text in a dark window.
- It shares the Telex's styles, so the two look like a matching pair.
- On phones and portrait tablets there's no room on the dash between the compass and the Telex, and the wheel covers the lower panel. So on screens narrower than about 6:5, it hangs from the ceiling over the middle window on brass straps instead.

The changes are in [shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html) (the instrument, the stored origin, the formatting helper, and the update in ``drawStation``) and

[shipstyle.css](Ship%20Functionality%20Demo/shipstyle.css) (placement and the narrow-screen layout). Nothing is committed.

Prompt: Now that we have this mechanic, we can track the player location. Next, I want to give the player a goal to reach on the beach. Since the visuals of the game don't allow them to "reach" the beach, it only allows them to get close to them. Thus, upon the game starting, create a random x,y location on the beach for the player to "reach". Show a visual of where that location is (this will be removed later, for now we are using it for debugging purposes).

Response: The game now picks a random spot on the beach each time it starts. The goal counts as reached once the ship gets within 600 m of it. That's the right threshold because the shoals stop the ship about 450–480 m from the shore, so it can never get any closer.

****Test results****

- ****Placement:**** in three normal loads the goal landed at a different spot on the beach each time. When the goal was in view, the pink flag stood on the shoreline where the readout said it would be.
- ****Reaching it:**** I made a scratch copy of the page (your files weren't changed) and started the ship right next to a fixed goal. The ship was pushed back to the shoal limit, 480 m from the goal, and the readout turned green and showed REACHED.
- I haven't steered a full run to the goal by hand, since that takes several minutes of sailing.

****The goal (this part stays)****

- It's placed somewhere between 500 m and 7,500 m north along the coast, 30 m up the sand from the water's edge. It uses the same X/Y system as the position indicator.
- Each frame, the game checks the ship's distance to the goal. Once it's within range, the goal is marked as reached and "[goal] reached" is logged to the console.
- Nothing else happens yet when the goal is reached. The check in `checkGoal()` is where to add what happens next, like a message or ending the round.
- The range limits and the 600 m threshold are set as constants in the coast section of [shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html), so they're easy to change.

****Debug visuals (to be removed later)****

- ****Flag:**** a pink flag on the beach with its distance, e.g. "GOAL 3880 m". It turns green once reached. It stays the same size at any distance, so you can see it from anywhere, and it also shows through the binoculars.
- ****Readout box (top left):**** the goal's X/Y, the distance and bearing to it, which way to turn (e.g. "46° to starboard"), and whether it's been reached.
- To remove them, delete everything marked `DEBUG`. That's the `drawGoalMarker()` and `drawDebug()` functions and their two calls, the `#debugGoal` element in the HTML, and the `.debug` styles in [shipstyle.css](Ship%20Functionality%20Demo/shipstyle.css).

If the goal falls more than about 35° to starboard of the starting heading, the flag starts out of view. The readout's bearing tells you which way to turn. Nothing is committed.

Prompt: After a paper on the Telex machine is printed, allow the player to "tear off" the message by pulling it away from the machine. After the paper is removed, it will reset the message interval, and print another random message.

Response: The teleprinter slip in the ship demo can now be torn off by dragging it away from the machine. Once it's gone, the message timer restarts and another random signal prints when it runs out. I tested it in Chrome and every step behaved as expected, with no console errors.

****How it works:****

- ****Pulling:**** drag the slip and it leans toward your hand, still stuck in the machine. If you let go before it tears, it springs back. A plain click still picks it up to read, as before.
- ****Tearing:**** once you've pulled it about 30px away, it comes free with a ragged bottom edge and follows the pointer. When you let go it falls and fades out. The lamp stops flashing if the message hadn't been read yet.
- ****Next message:**** tearing starts the usual 40–120 second wait, then a new random message prints.
- ****Keyboard:**** pressing Up while the slip is selected tears it straight off. That way you don't need a mouse, and the slip's screen-reader label says so.

****One behaviour change:**** closing a message after reading it ("Put it down") no longer starts the timer. The slip stays in the machine, and the next message only comes once it's torn off. So a player who never tears the paper won't get any more messages. That's how I read your request, but it's easy to change if you'd rather keep a fallback.

The changes are in [shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html) and [shipstyle.css](Ship%20Functionality%20Demo/shipstyle.css). You can set how far the paper has to be pulled with `TEAR\_PULL` at [shipbuild-demo.html:556](Ship%20Functionality%20Demo/shipbuild-demo.html#L556).

I only changed the ship demo, since that's the version with a message timer. The standalone `Telex Machine Demo` is untouched: it has no timer, and it clears paper by clicking the roller.

Prompt: After a paper on the Telex machine is printed, allow the player to "tear off" the message by pulling it away from the machine. After the paper is removed, it will reset the message interval, and print another random message.

Response: The teleprinter slip in the ship demo can now be torn off by dragging it away from the machine. Once it's gone, the message timer restarts and another random signal prints when it runs out. I tested it in Chrome and every step behaved as expected, with no console errors.

****How it works:****

- ****Pulling:**** drag the slip and it leans toward your hand, still stuck in the machine. If you let go before it tears, it springs back. A plain click still picks it up to read, as before.
- ****Tearing:**** once you've pulled it about 30px away, it comes free with a ragged bottom edge and follows the pointer. When you let go it falls and fades out. The lamp stops flashing if the message hadn't been read yet.
- ****Next message:**** tearing starts the usual 40–120 second wait, then a new random message prints.
- ****Keyboard:**** pressing Up while the slip is selected tears it straight off. That way you don't need a mouse, and the slip's screen-reader label says so.

****One behaviour change:**** closing a message after reading it ("Put it down") no longer starts the timer. The slip stays in the machine, and the next message only comes once it's torn off. So a player who never tears the paper won't get any more messages. That's how I read your request, but it's easy to change if you'd rather keep a fallback.

The changes are in [shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html) and [shipstyle.css](Ship%20Functionality%20Demo/shipstyle.css). You can set how far the paper has to be pulled with `TEAR\_PULL` at [shipbuild-demo.html:556](Ship%20Functionality%20Demo/shipbuild-demo.html#L556).

I only changed the ship demo, since that's the version with a message timer. The standalone `Telex Machine Demo` is untouched: it has no timer, and it clears paper by clicking the roller.

Prompt: After pulling it off, allow the player to place it anywhere on the ship.

Response: Torn-off slips now stay wherever the player drops them, and they can be picked up again to read or moved around. I tested it in Chrome and every step worked, with no console errors.

- ****Placing:**** once a slip tears free, it follows the pointer and stays where you let go, tilted at the angle you were holding it. It can go anywhere on the helm, including the dash, windscreen, window frames or wheel. It can't go off-screen or up under the ceiling beams.
- ****Reading:**** click a slip that's been put down and it opens full size, the same way as when it's in the machine. "Put it down" puts it back where it was lying.
- ****Moving:**** drag a slip to move it again. A short click still reads it rather than moving it. The slip you last touched sits on top of any others.
- ****Keyboard:**** pressing Up on the slip in the machine tears it off and lays it just beside the teleprinter. After that, the arrow keys move a selected slip and Enter reads it.
- ****Screen resizing:**** positions and sizes are stored relative to the helm, so slips stay in the same spot on the ship when the window changes size.
- ****Next message:**** tearing still starts the wait for the next signal, as before.

Slips build up over a game and there's no way to throw one away yet. They also sit above the wheel, so a slip dropped on it will cover part of it until it's moved. If you want, I can add a way to discard slips, such as dropping one off the edge of the screen.

The changes are in

[shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html), where the torn slips get their own layer inside the helm, and
[shipstyle.css](Ship%20Functionality%20Demo/shipstyle.css).

Prompt: Change the end goal to no longer be dynamic, but instead a single location on the beach that will take 10 minutes for the player to travel.

Response: The goal is now in the same place every game, at 4,505 m north along the beach, instead of being picked at random each time.

At 4,505 m north the goal sits at about (+2,666, +4,585) on the position indicator. From the start, the boat has to travel about 4,620 m to get within the 600 m reach radius. At the current `SPEED = 7.7` m/s that takes 600 seconds, or 10 minutes, in a straight line. That's calculated, not timed in a playthrough.

In practice it will take a little longer than 10 minutes, since the player has to spot the goal and steer toward it rather than head straight there from the start. If you want the real playing time to come out at 10 minutes, move the goal a bit closer (about 4,000 m north is roughly 9 minutes in a straight line).

There are two edits in

[shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html):

- [Line 609](Ship%20Functionality%20Demo/shipbuild-demo.html#L609): `GOAL\_ALONG` is now a single value, `4505`, instead of the `[500, 7500]` range, and the comment above it no longer says the goal is picked fresh each game.

- [Line 960](Ship%20Functionality%20Demo/shipbuild-demo.html#L960): `placeGoal()` uses that fixed value instead of `between(GOAL\_ALONG)`.

This location is tied to the current settings. If you change `SPEED`, the start position or `GOAL\_REACH`, the goal will no longer be exactly 10 minutes away and will need moving.

Prompt: @"Telex Machine Demo/signals/01-sailing-orders.js"

@"Telex Machine Demo/signals/02-route-x.js"

@"Telex Machine Demo/signals/03-the-beaches.js"

@"Telex Machine Demo/signals/04-air-attack.js"

@ "Telex Machine Demo/signals/05-homeward.js"

@ "Telex Machine Demo/signals/06-recall.js"

Rework these javascript files carrying messages for the player to receive throughout the journey. Edit their language to be more guiding to the player. Each page should include a degree amount that the wheel must be rotated towards in order to head in the direction of the goal. However, this direction should not lead straight to the goal, and instead adjust the player in varying directions that eventually end up at the end goal. If needed, make a separate JS file separate from these signal files, so that the pages can calculate how many degrees the player must turn, and display that into the message.

Response: Every signal now gives a course that steers the player along a zig-zag route to the beach. I tested it in the actual ship: the harness steered the real wheel by whatever each slip said, and the ship logged "goal reached" after about 14½ minutes of game time.

****How it works****

- ****New file, [course.js](Telex%20Machine%20Demo/course.js):**** it holds the route as five named marks, measured in the same metres as the ship's POSITION gauge. The legs run 000 → 339 → 059 → 355 → 062, and only the last one points at the goal. All the marks stay well clear of the shoals.

- ****Signal text has placeholders:**** `{HELM}`, `{COURSE}`, `{MARK}` and `{MINUTES}`. When a slip prints, they're replaced using the boat's position and heading at that moment. For example: ****"TURN 80 DEGREES TO PORT. PUT THE WHEEL OVER TO THE LEFT AND HOLD IT UNTIL THE COMPASS READS 352, THEN CENTRE THE WHEEL. 352 BRINGS YOU TO THE ZUYDCOOTE PASS IN ABOUT 3 MINUTES."****

- ****The degree amount is a turn plus a compass reading:**** in the game the wheel sets the rudder, and the boat keeps turning until the wheel is centred. So a signal can't say "turn the wheel to 062". Instead it gives how far to turn and which way, the compass reading to stop at, and a reminder to centre the wheel.

- ****Each course is worked out from where the boat actually is:**** once the boat passes a mark, the next slip gives the next leg. If the player drifts, the next slip corrects them.

- ****I rewrote all six signals**** to be plainer and more guiding while still passing the rulebook's five checks. Every course is three figures, every change of course gives a reason, and there are no "hurry" words or wireless requests.

- ****Small wiring changes:**** the ship's `compose()` in [shipbuild-demo.html](Ship%20Functionality%20Demo/shipbuild-demo.html) now passes in the boat's position and heading, and loads `course.js`. The same tweak went into [telex.js](Telex%20Machine%20Demo/telex.js) and its `index.html`. Opened on its own, the teleprinter has no boat to read, so it steps through the route one leg per print.

****Tests****

- Opened on its own, the teleprinter printed all six slips correctly.

- A simulation of 800 voyages using the ship's physics reached the goal every time, including ones with a slow, sloppy helmsman.

****Things you should know****

1. ****Signals arrive at random in the ship,**** so no file is tied to a leg; whichever one prints gives the current course. The catch is that serial numbers and times arrive out of order and can repeat. The rulebook tells players that's the sign of a forgery. I'd suggest having the ship deliver signals in order, which is a small change. Want me to make it?
2. ****The trip is longer than the 10-minute target.**** A careful helmsman takes a median of 13.7 minutes, against 11.3 for the straight line. Moving the marks in `course.js` shortens it, and the signals follow automatically.
3. ****I removed `ANSWER: 190` from `03-the-beaches.js`,**** because a fixed course can't be right when it depends on where the boat is. I kept `UNLOCKS` and updated the telex README to match.
4. ****I kept the filenames of `05-homeward` and `06-recall`**** but reworded them for the outbound trip. 05 is now about loaded ships coming home along the channel, and 06 is the last run before withdrawing.

Nothing is committed.
